Лабораторной работе №2
«Диаграммы классов, последовательностей, объектов»

1. Цель работы

Целью лабораторной работы является изучение принципов объектно-ориентированного проектирования и освоение средств визуального моделирования программного обеспечения с использованием UML-диаграмм.

В ходе выполнения лабораторной работы необходимо разработать диаграмму классов, диаграммы последовательностей и диаграмму объектов для выбранной предметной области. Также необходимо реализовать спроектированную систему на языке программирования C# и подготовить документацию с пояснениями к каждой диаграмме.

2. Постановка задачи

В рамках лабораторной работы необходимо спроектировать программную систему для выбранной предметной области.

Для описания архитектуры системы используются UML-диаграммы, позволяющие представить структуру классов, взаимодействие объектов и конкретные экземпляры элементов системы.

В соответствии с заданием необходимо выполнить следующие действия:

1. Разработать диаграмму классов, содержащую не менее шести классов, включая абстрактный класс или интерфейс.
2. Предусмотреть не менее двух классов-наследников абстрактного класса или реализаций интерфейса.
3. Использовать не менее четырех различных видов связей между элементами диаграммы.
4. Разработать три диаграммы последовательностей для различных сценариев работы системы.
5. Создать диаграмму объектов, содержащую не менее четырех объектов.
6. Реализовать спроектированную систему на языке C#.
7. Подготовить единую документацию с текстовым описанием каждой диаграммы.

3. Описание предметной области

В качестве предметной области рассматривается программная система «Трансфер футболистов».

В процессе работы система использует несколько взаимосвязанных классов, каждый из которых отвечает за определенную часть функциональности. Такое разделение позволяет упростить сопровождение программного кода, повысить его читаемость и обеспечить возможность дальнейшего расширения функциональности.

Для построения архитектуры применяется объектно-ориентированный подход, основанный на инкапсуляции, наследовании и полиморфизме.

4. Разработка UML-диаграммы классов

Диаграмма классов предназначена для отображения статической структуры программной системы. Она описывает классы, их атрибуты, методы и отношения между ними.

В разработанной системе выделены основные классы, обеспечивающие реализацию функциональности приложения.

При проектировании используются следующие элементы:
- обычные классы;
- абстрактный класс или интерфейс;
- классы-наследники;
- ассоциации;
- зависимости;
- агрегация или композиция;
- наследование или реализация интерфейса.

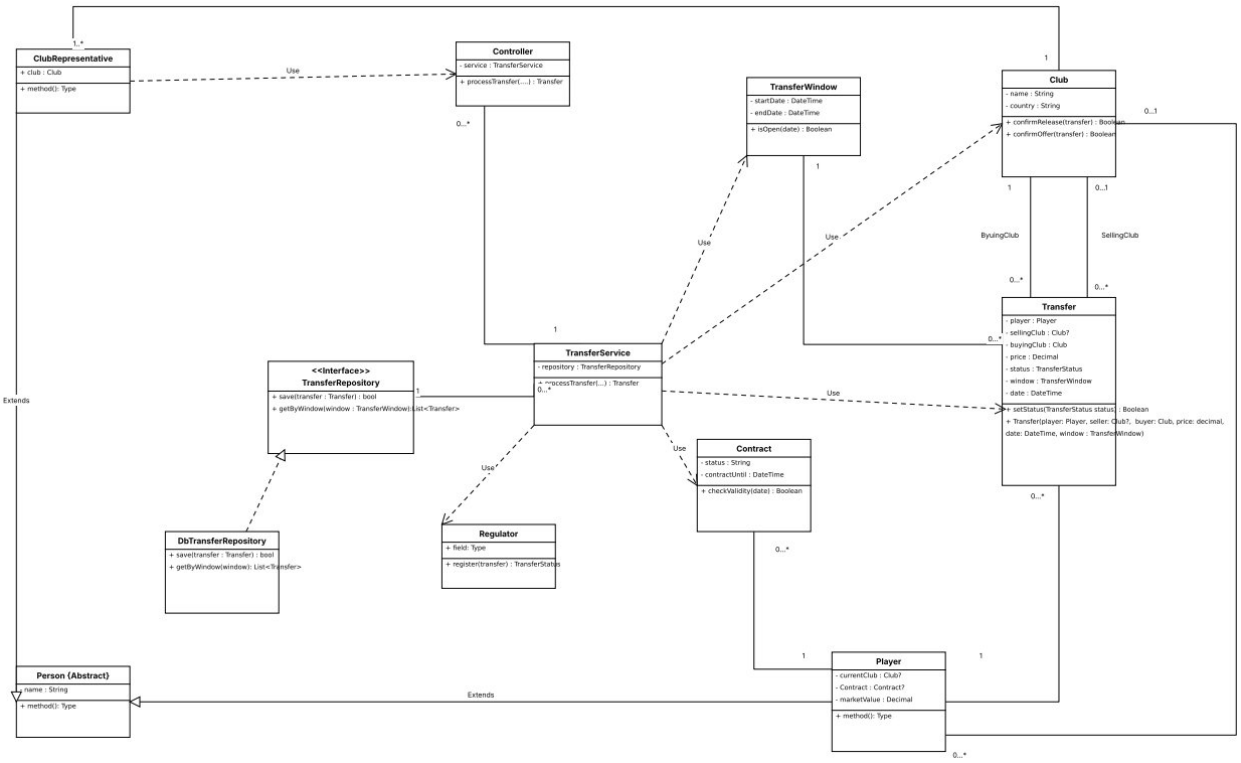


Рисунок 1 — UML-диаграмма классов программной системы

На диаграмме представлены основные классы программной системы, их атрибуты, методы и взаимосвязи.

Абстрактный класс (или интерфейс) определяет общую структуру и поведение для производных классов. Конкретные классы реализуют необходимую функциональность и обеспечивают специализацию базового поведения.

Связи между классами позволяют отразить взаимодействие элементов системы, а также определить зависимости между отдельными компонентами программной архитектуры.

Использование наследования позволяет повторно использовать общую функциональность, а применение интерфейсов обеспечивает гибкость архитектуры и возможность расширения системы.

5. Разработка диаграмм последовательностей

Диаграммы последовательностей используются для отображения взаимодействия объектов во времени. Они показывают порядок отправки сообщений, вызовы методов и последовательность выполнения операций.

Для проектируемой системы рассматриваются три основных сценария работы.

5.1. Диаграмма последовательности реального сценария работы

Данная диаграмма описывает процесс выполнения одной из основных операций системы.

В рамках сценария пользователь инициирует определенное действие. После этого система получает запрос, выполняет необходимые проверки и передает управление соответствующим объектам.

В результате взаимодействия объектов выполняется необходимая операция, после чего пользователю возвращается результат.

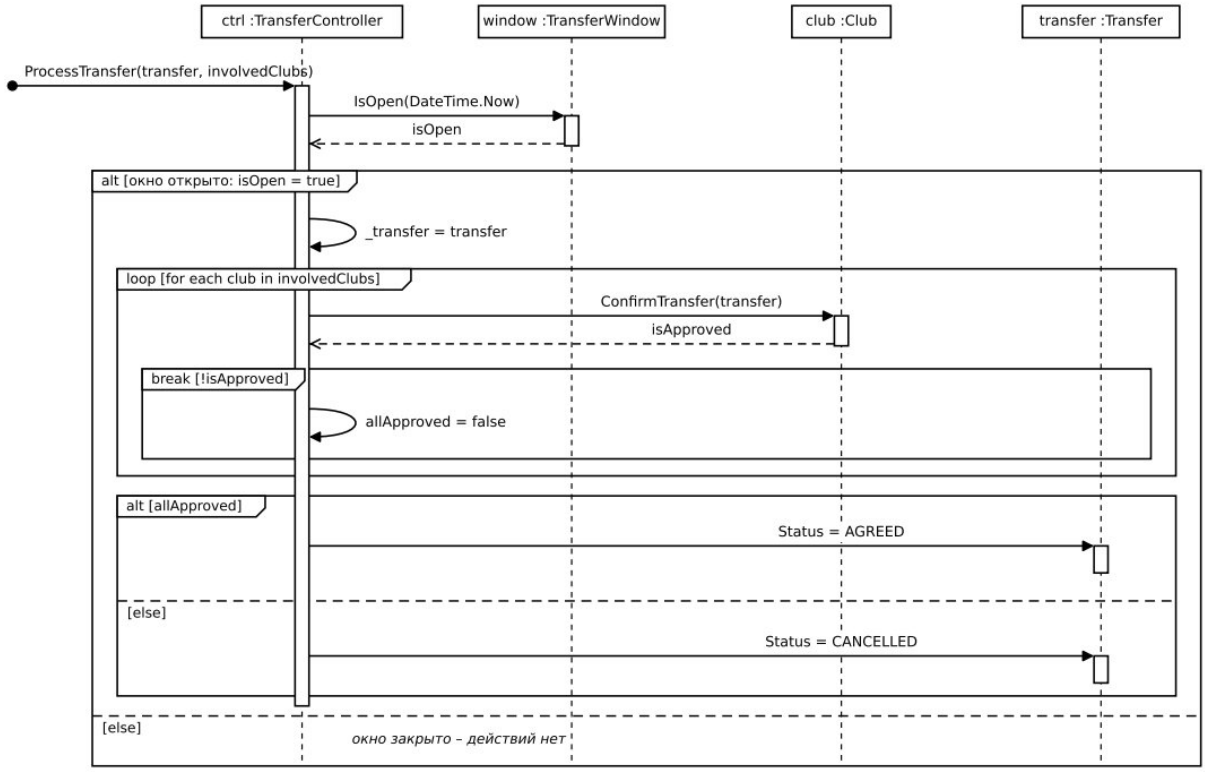


Рисунок 2 — Диаграмма последовательности основного сценария работы системы

Диаграмма позволяет проследить порядок взаимодействия между пользователем и объектами системы, определить последовательность вызовов методов и понять, каким образом выполняется основная операция.

5.2. Диаграмма последовательности вложенных активностей

Вторая диаграмма отражает процесс выполнения операции, состоящей из нескольких последовательных или вложенных действий.

В ходе выполнения сценария один объект инициирует операцию, которая приводит к вызову дополнительных методов других объектов. Каждый объект выполняет собственную часть задачи и передает результат следующему участнику взаимодействия.

Такой подход позволяет разделить сложную операцию на несколько более простых этапов.

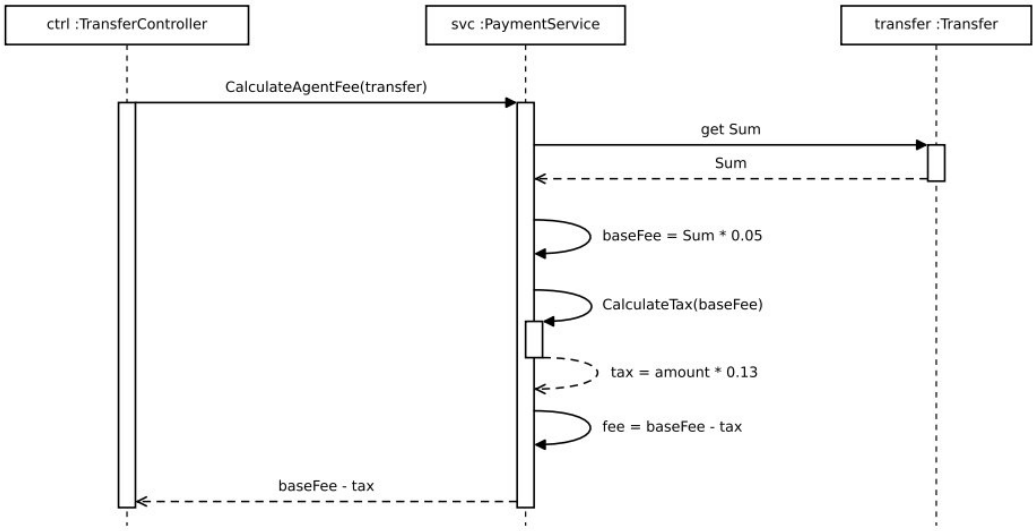


Рисунок 3 — Диаграмма последовательности вложенных активностей

Диаграмма демонстрирует распределение ответственности между объектами и последовательность выполнения внутренних операций. Она позволяет определить зависимости между действиями и проследить передачу управления в процессе выполнения сценария.

5.3. Диаграмма последовательности создания объекта

Третья диаграмма описывает процесс создания нового объекта в программной системе.

При выполнении данного сценария инициируется операция создания объекта, после чего вызывается соответствующий конструктор. Создаваемый объект получает необходимые начальные данные и становится доступным для дальнейшего использования другими элементами системы.

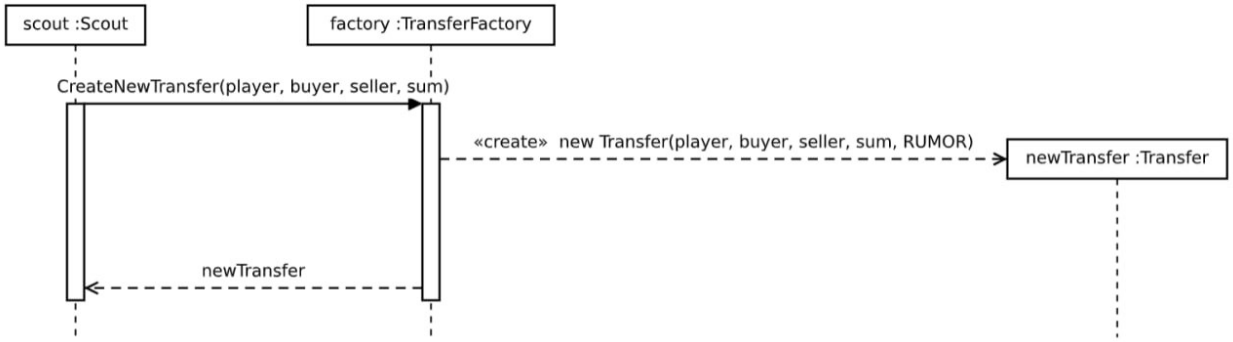


Рисунок 4 — Диаграмма последовательности создания объекта

Данная диаграмма позволяет проследить процесс инициализации нового объекта, определить участвующие в создании элементы системы и понять, каким образом осуществляется передача необходимых параметров.

6. Разработка диаграммы объектов

Диаграмма объектов предназначена для отображения конкретных экземпляров классов и связей между ними в определенный момент времени.

В отличие от диаграммы классов, которая описывает общую структуру системы, диаграмма объектов демонстрирует конкретные объекты с заполненными значениями атрибутов.

В рамках разработанной системы диаграмма объектов показывает взаимодействие нескольких экземпляров классов. Каждый объект представляет собой отдельный элемент системы, обладающий собственным состоянием и набором характеристик.

Связи между объектами отражают отношения и взаимодействия, возникающие в процессе работы приложения.

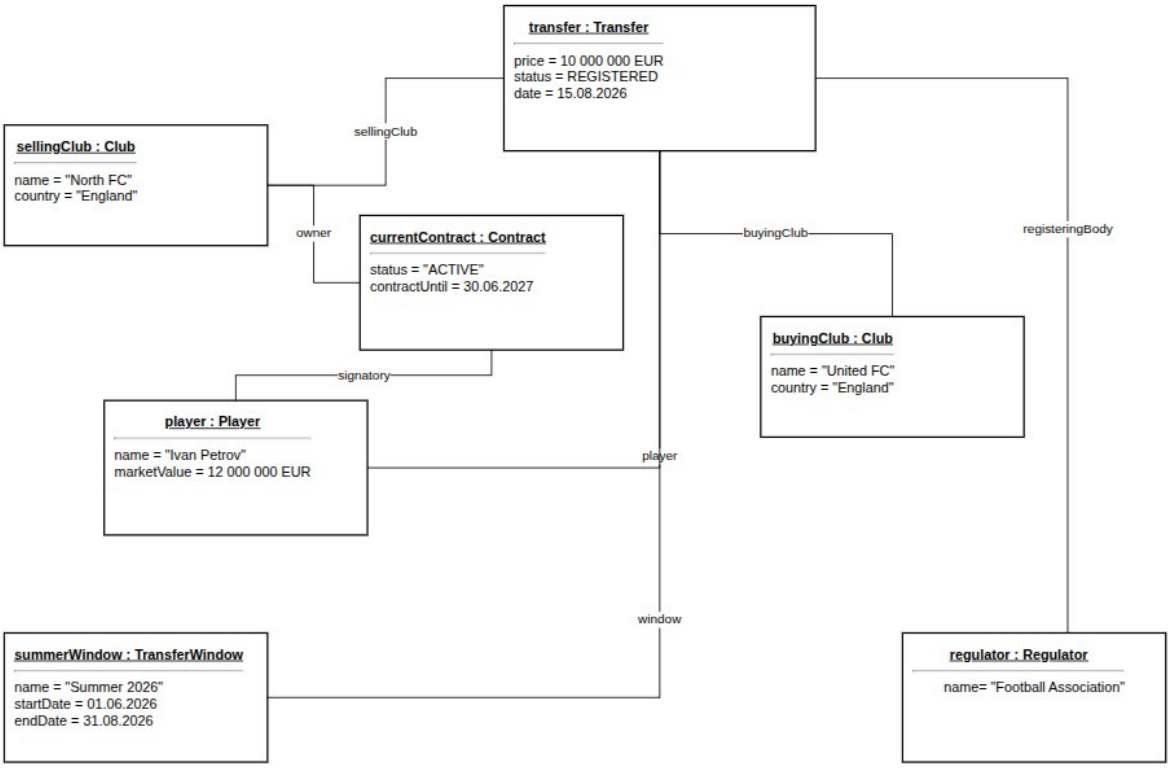


Рисунок 5 — UML-диаграмма объектов программной системы

Диаграмма позволяет наглядно представить структуру взаимодействия конкретных экземпляров классов и проследить связи между ними.

В качестве объектов используются экземпляры классов, представленных на диаграмме классов. Каждый объект имеет определенные значения атрибутов, характеризующие его текущее состояние.

7. Реализация программной системы на языке C#

После проектирования UML-диаграмм была выполнена реализация программной системы на языке программирования C#.

При разработке использовались принципы объектно-ориентированного программирования. Основная функциональность распределена между отдельными классами, каждый из которых отвечает за определенные задачи.

В программной реализации используются:

- классы и объекты;

- конструкторы;
- методы и свойства;
- инкапсуляция данных;
- наследование;
- полиморфизм;
- взаимодействие между объектами.

Структура программного кода соответствует разработанным UML-диаграммам. Классы, представленные на диаграмме классов, реализованы в исходном коде, а взаимодействия между объектами соответствуют сценариям, отображенным на диаграммах последовательностей.

7.1 Структура проекта

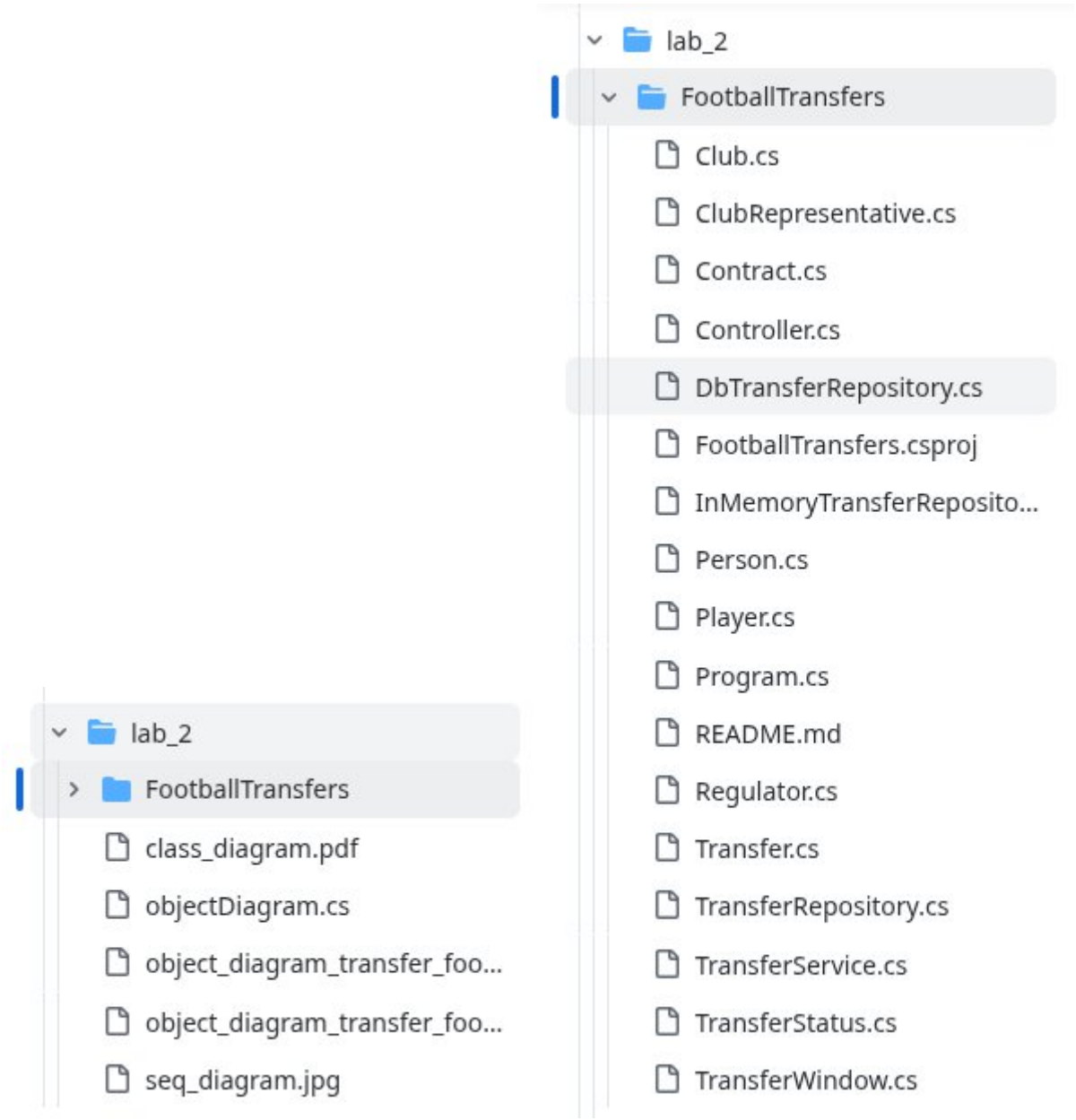


Рисунок 6 — Структура проекта программной системы

В проекте выделены отдельные файлы для реализации классов и основных элементов приложения. Такое разделение обеспечивает удобство разработки и упрощает дальнейшее сопровождение программного кода.

8. Документирование UML-диаграмм

Для обеспечения понятности проектной документации каждая диаграмма сопровождается текстовым описанием.

Документирование необходимо для того, чтобы разные участники разработки могли правильно интерпретировать представленную архитектуру и использовать диаграммы в дальнейшей работе.

В описании каждой диаграммы выделяются следующие характеристики:

- вопрос, на который отвечает схема;
- предполагаемый читатель;
- действие, которое читатель сможет выполнить после изучения схемы;
- информация, представленная на схеме;
- информация, которая не входит в область рассмотрения;
- допущения, используемые при проектировании.

8.1. Документация диаграммы классов

Таблица 2 — Текстовое описание диаграммы классов

Поле	Что пишем
Вопрос	Из каких классов состоит система и как они связаны между собой?
Читатель	Разработчик программного обеспечения
Что сделает читатель	Определит структуру системы, основные классы и отношения между ними
Что показано на схеме	Классы, атрибуты, методы, наследование, ассоциации и другие связи
Что не показано	Порядок выполнения операций, алгоритмы методов и конкретные значения объектов
Допущения	Предполагается, что представленные классы и связи соответствуют проектируемой архитектуре системы

8.2. Документация первой диаграммы последовательности

Таблица 3 — Текстовое описание первой диаграммы последовательности

Поле	Что пишем
Вопрос	Как выполняется основной сценарий работы системы?
Читатель	Разработчик или тестировщик
Что сделает читатель	Проследит последовательность действий и проверит правильность взаимодействия объектов
Что показано на схеме	Участники взаимодействия, сообщения и последовательность вызовов методов
Что не показано	Внутренняя реализация алгоритмов и структура всех классов
Допущения	Сценарий рассматривается при условии корректного выполнения входных операций

8.3. Документация второй диаграммы последовательности

Таблица 4 — Текстовое описание второй диаграммы последовательности

Поле	Что пишем
Вопрос	Каким образом выполняются вложенные действия при обработке операции?
Читатель	Разработчик программного обеспечения
Что сделает читатель	Определит порядок выполнения внутренних операций и взаимодействие объектов
Что показано на схеме	Вложенные вызовы методов, участников взаимодействия и передачу управления
Что не показано	Подробная реализация внутренних алгоритмов каждого метода
Допущения	Вызовы методов происходят в соответствии с предусмотренной логикой приложения

8.4. Документация третьей диаграммы последовательности

Таблица 5 — Текстовое описание третьей диаграммы последовательности

Поле	Что пишем
Вопрос	Как происходит создание и инициализация нового объекта?
Читатель	Разработчик программного обеспечения
Что сделает читатель	Проследит процесс создания объекта и передачу начальных параметров
Что показано на схеме	Инициатор создания, вызов конструктора, передача параметров и создание экземпляра
Что не показано	Детали управления памятью и внутренние особенности среды выполнения
Допущения	Создание объекта выполняется при наличии необходимых исходных данных

8.5. Документация диаграммы объектов

Таблица 6 — Текстовое описание диаграммы объектов

Поле	Что пишем
Вопрос	Какие конкретные объекты существуют в системе и как они связаны?
Читатель	Разработчик или тестировщик
Что сделает читатель	Определит конкретные экземпляры классов и связи между ними
Что показано на схеме	Экземпляры классов, значения их атрибутов и отношения между объектами
Что не показано	Полная структура классов и последовательность выполнения методов
Допущения	Значения атрибутов представлены для выбранного состояния системы

9. Заключение

В ходе выполнения лабораторной работы №2 были рассмотрены основные принципы объектно-ориентированного проектирования и средства визуального моделирования программного обеспечения.

В процессе работы были разработаны UML-диаграммы, позволяющие описать структуру программной системы, взаимодействие между объектами и конкретные экземпляры классов.

Была рассмотрена диаграмма классов, отображающая основные элементы архитектуры и связи между ними. Также были разработаны диаграммы последовательностей, демонстрирующие порядок выполнения различных операций и взаимодействие объектов во времени.

Дополнительно была создана диаграмма объектов, позволяющая представить конкретное состояние системы и отношения между экземплярами классов.

На основании разработанных диаграмм была выполнена программная реализация на языке C#. В результате были закреплены навыки использования классов, объектов, наследования, инкапсуляции и полиморфизма.

Таким образом, цель лабораторной работы достигнута, а поставленные задачи выполнены.